

Basics of Applied Numerical Analysis – HW1

Anastasia Golubkova

4, October 2026

The code for the problems (which is also typed here) is available here: <https://github.com/Nastya-golubkova/BoANA/blob/main/hw1/hw1.ipynb>

Problem 1. Compute π -number using the recursive program for the following formula through generalized continuous fractions:

$$\pi = \frac{4}{1 + \frac{1^2}{3 + \frac{2^2}{5 + \frac{3^2}{7 + \dots}}}}$$

Begin with obtaining the “numerically exact” solution comparing results with n and $n + 1$ terms, until error stops decreasing (show both absolute and relative errors). Show linear convergence with number of terms, comparing with “numerically exact” solution and between successive approximations. Use double logarithmic plot (logarithm of both coordinates). Why are you supposed to get a linear function? What will happen if the rate of convergence will be a quadratic one? Why linear dependence is observed only on a finite interval?

Solution.

```
1 import math
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def compute_pi(n):
6     frac = float(2 * n - 1)
7     for k in range(n - 1, 0, -1):
8         frac = (2 * k - 1) + k**2 / frac
9     return 4.0 / frac
10
11 N = 30
12 n = np.arange(1, N + 1)
13 pi_n = np.array([compute_pi(i) for i in n])
14 pi_exact = math.pi
15
16
17 abs_error = np.abs(pi_n - pi_exact)
18 rel_error = abs_error / abs(pi_exact)
19
20
21 successive_diff = np.abs(np.diff(pi_n))
22
23
24 k_stop = np.argmin(successive_diff)
25 print("Numerically exact solution:")
26 print(f"pi = {pi_exact:.16f}")
27 print(f"Convergence stops at n = {k_stop + 1}")
28 print(f"|pi_{n+1} - pi_n| = {successive_diff[k_stop]:.3e}")
29 print()
30 print(" n      pi_n      abs error      rel error")
31 for i in range(N):
32     print(f"{i+1:4d} {pi_n[i]:.15f} {abs_error[i]:.3e} {rel_error[i]:.3e}")
33
34
35 eps = abs_error[abs_error > 0]
```

```

36 plt.figure(figsize=(7, 5))
37 plt.loglog(eps[:-1], eps[1:], "o-", markersize=4)
38 plt.xlabel(r"$\varepsilon_n$")
39 plt.ylabel(r"$\varepsilon_{n+1}$")
40 plt.title("Log-log: consecutive errors $\varepsilon_{n+1}$ vs $\varepsilon_n$")
41 plt.grid(True, which="both", linestyle=":")
42 plt.tight_layout()
43 plt.show()
44
45
46 diffs = successive_diff[successive_diff > 0]
47 plt.figure(figsize=(7, 5))
48 plt.loglog(diffs[:-1], diffs[1:], "s-", markersize=4)
49 plt.xlabel(r"$|\pi_n - \pi_{n-1}|$")
50 plt.ylabel(r"$|\pi_{n+1} - \pi_n|$")
51 plt.title("Log-log: successive approximations")
52 plt.grid(True, which="both", linestyle=":")
53 plt.tight_layout()
54 plt.show()
55
56
57 Numerically exact solution:
58 pi = 3.1415926535897931
59 Convergence stops at n = 22
60 |pi_{n+1} - pi_n| = 0.000e+00
61
62 n      pi_n      abs error      rel error
63 1  4.000000000000000 8.584e-01 2.732e-01
64 2  3.000000000000000 1.416e-01 4.507e-02
65 3  3.166666666666667 2.507e-02 7.981e-03
66 4  3.137254901960785 4.338e-03 1.381e-03
67 5  3.142342342342342 7.497e-04 2.386e-04
68 6  3.141463414634146 1.292e-04 4.114e-05
69 7  3.141614906832298 2.225e-05 7.083e-06
70 8  3.14158825092124 3.828e-06 1.219e-06
71 9  3.141593311879928 6.583e-07 2.095e-07
72 10 3.141592540446540 1.131e-07 3.601e-08
73 11 3.141592673030334 1.944e-08 6.188e-09
74 12 3.141592650250245 3.340e-09 1.063e-09
75 13 3.141592654163366 5.736e-10 1.826e-10
76 14 3.141592653491295 9.850e-11 3.135e-11
77 15 3.141592653606706 1.691e-11 5.384e-12
78 16 3.141592653586889 2.904e-12 9.243e-13
79 17 3.141592653590292 4.987e-13 1.587e-13
80 18 3.141592653589707 8.571e-14 2.728e-14
81 19 3.141592653589808 1.465e-14 4.665e-15
82 20 3.141592653589791 2.220e-15 7.068e-16
83 21 3.141592653589794 4.441e-16 1.414e-16
84 22 3.141592653589793 0.000e+00 0.000e+00
85 23 3.141592653589793 0.000e+00 0.000e+00
86 24 3.141592653589793 0.000e+00 0.000e+00
87 25 3.141592653589793 0.000e+00 0.000e+00
88 26 3.141592653589793 0.000e+00 0.000e+00
89 27 3.141592653589793 0.000e+00 0.000e+00
90 28 3.141592653589793 0.000e+00 0.000e+00
91 29 3.141592653589793 0.000e+00 0.000e+00
92 30 3.141592653589793 0.000e+00 0.000e+00

```

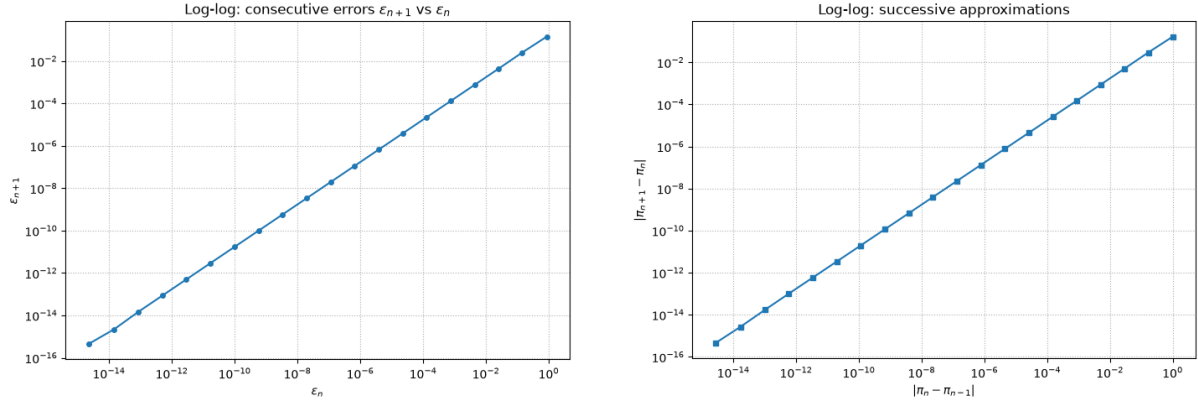


Рис. 1: Left: $\log(\text{error}_n)$ vs $\log(\text{error}_{n+1})$; Right: $|\pi(x_i) - \pi(x_{i+1})|$ vs $|\pi(x_i) - \pi(x_{i-1})|$.

1. **Why linear?** For this continued fraction the convergence is linear, i.e. geometric: the ratio of successive errors tends to a constant $q < 1$,

$$\varepsilon_{n+1} \approx q \varepsilon_n.$$

On the log-log plot of ε_{n+1} vs ε_n this gives

$$\log \varepsilon_{n+1} = \log q + 1 \cdot \log \varepsilon_n,$$

which is a straight line with slope 1. This is exactly what is observed on both graphs (errors against the exact π and successive differences), confirming linear convergence.

The constant can be read off from the vertical offset: $q \approx 0.1716 \approx 3 - 2\sqrt{2}$.

2. **What if the convergence were quadratic?** If $\varepsilon_{n+1} \sim C \varepsilon_n^2$, then

$$\log \varepsilon_{n+1} = \log C + 2 \log \varepsilon_n,$$

so the log-log plot of ε_{n+1} vs ε_n would be a straight line with slope 2 instead of 1. The number of correct digits would double at every step, and machine precision would be reached after only a few iterations.

3. **Why is the linear dependence observed only on a finite interval?**

- For small n the asymptotic regime has not yet been established: there is a transient, and the ratio $\varepsilon_{n+1}/\varepsilon_n$ is not yet equal to q .
- For large n (here $n \approx 20$) the error reaches the level of machine precision, $\sim 10^{-16}$ for double precision. Rounding errors dominate, the difference $\pi_n - \pi$ stops decreasing and fluctuates around this level (this is also where $|\pi_{n+1} - \pi_n|$ stops decreasing, which gives the numerically exact solution).

Problem 2. Convert the following binary (i.e. base-2) numbers to decimal:

$$1011.\overline{010}_2, \quad 11.\overline{10101}_2$$

Solution.

$$\begin{aligned} x &= 1011.\overline{010}_2, \\ 2^3 \cdot x &= 1011010.\overline{010}_2, \\ 2^3 \cdot x - x &= x(2^3 - 1) = 1011010_2 - 1011_2 = 1001111_2 = 79, \\ x &= \frac{79}{7} = 11\frac{2}{7}. \end{aligned}$$

$$\begin{aligned} x &= 11.\overline{10101}_2, \\ 2^5 \cdot x &= 1110101.\overline{10101}_2, \\ 2^5 \cdot x - x &= x(2^5 - 1) = 1110101_2 - 11_2 = 1110010_2 = 114, \\ x &= \frac{114}{31}. \end{aligned}$$

Problem 3. Sauer, page 16, Exercise 16 (a,b).

Find the IEEE double precision representation $fl(x)$, and find the exact difference $fl(x) - x$ for the given real numbers. Check that the relative rounding error is no more than $\epsilon_{\text{mach}}/2$.

$$(\cdot) \quad x = 2.75$$

(.) $x = 2.7$

Solution.

(a) $x = 2.75$
 $x = 10.11_2$
 $x \cdot 2^{-1} = 1.011_2 = x_{\text{norm}}$

[illegible]

DP representation : 0 10000000000 011000

Since 2.75 is exactly representable in binary,

$$fl(x) = x = 2.75,$$

so

$$fl(x) - x = 0, \quad \frac{|fl(x) - x|}{|x|} = 0.$$

(b) $x = 2.7$

$$0.7 \cdot 2 = 1.4$$

$$0.4 \cdot 2 = 0.8$$

$$0.8 \cdot 2 = 1.6$$

$$0.6 \cdot 2 = 1.2$$

$$0.2 \cdot 2 = 0.4$$

Therefore,

$$0.7_{10} = 0.1\overline{0110}_2,$$

and hence

$$x = 2.7 = 10.1\overline{0110}_2.$$

Normalizing,

$$x = 1.01\overline{0110}_2 \cdot 2^1.$$

01011001100110011001100110011001100110011001,
and the next bit $b_{53} = 1$. Since there are nonzero bits after b_{53} , we round up:

Thus,

```
exponent = 1  
mantissa = 010110011001100110011001100110011001100110011010  
sign = 0  
bias exponent = 10000000000
```

DP representation : 0 1000000000 010110011001100110011001100110011001100110011010

$$x = 10.1\overline{0110}_2 = 1.01\overline{0110}_2 \cdot 2^1, \quad fl(x) = 1.01011001100110011001100110011001100110011010_2 \cdot 2^1.$$

The last bit of the mantissa has weight $2^{-52} \cdot 2^1 = 2^{-51}$. The bits cut off are $b_{53}b_{54} \dots = \overline{1001}_2$, so the chopping error is

$$x - x_{\text{chop}} = 0.\overline{1001}_2 \cdot 2^{-52} \cdot 2^1 = \frac{3}{5} \cdot 2^{-51} = 0.6 \cdot 2^{-51}.$$

Since this is more than half an ulp, we round up:

$$fl(x) = x_{\text{chop}} + 2^{-51} = x - 0.6 \cdot 2^{-51} + 2^{-51},$$

$$fl(x) - x = (1 - 0.6) \cdot 2^{-51} = 0.4 \cdot 2^{-51} = \frac{2^{-50}}{5} \approx 1.776 \times 10^{-16}.$$

Relative error:

$$\frac{|fl(x) - x|}{|x|} = \frac{0.4 \cdot 2^{-51}}{2.7} = \frac{4}{27} \cdot 2^{-51} = \frac{16}{27} \cdot 2^{-53} \approx 6.58 \times 10^{-17}.$$

Since $\epsilon_{\text{mach}} = 2^{-52}$, we have $\epsilon_{\text{mach}}/2 = 2^{-53} \approx 1.11 \times 10^{-16}$, and

$$\frac{|fl(x) - x|}{|x|} = \frac{16}{27} \cdot 2^{-53} < 2^{-53} = \frac{\epsilon_{\text{mach}}}{2}$$

Problem 4. Sauer, page 19, Computer Problem 5. Consider a right triangle whose legs are of length 3344556600 and 1.2222222. How much longer is the hypotenuse than the longer leg? Give your answer with at least four correct digits.

```
1 import math
2
3 a = 3344556600
4 b = 1.2222222
5
6 difference = b ** 2 / (math.sqrt(a**2 + b**2) + a)
7
8 print(difference)
9
10 # output 2.2332214473105944e-10
```

Problem 5. Use the bisection method to compute a positive real number x satisfying $\sinh x = \cos x$ with a tolerance of 10^{-8} . List your initial interval in the case of bisection and the number of required iterations. Also print at least nine digits of the approximate root.

```

1 a = 0
2 b = 1
3 iterations = 0
4 def f(x):
5     return math.sinh(x) - math.cos(x)
6
7
8 m = (a + b) / 2
9 while (b - a) / 2 > 1e-8:
10     m = (a + b) / 2
11
12     if f(m) < 0:
13         a = m
14     else:
15         b = m
16
17     iterations += 1
18
19 print(f"x = {m}, f(x) = {f(m)}, iterations = {iterations}")
20
21 # output: x = 0.703290656208992, f(x) = -5.056139507075841e-09, iterations = 26

```

Problem 6. Assume that g is a continuously differentiable function and that the Fixed-Point Iteration $g(x)$ has exactly three fixed points, -3 , 1 , and 2 . Assume that $g'(-3) = 2.4$ and that FPI started sufficiently near the fixed point 2 converges to 2 . Find $g'(1)$.

Solution.

Since -3 , 1 , and 2 are the only fixed points of g , we have

$$g(-3) = -3, \quad g(1) = 1, \quad g(2) = 2.$$

Let

$$h(x) = g(x) - x.$$

Then the fixed points of g are exactly the zeros of h , so

$$h(-3) = h(1) = h(2) = 0.$$

Moreover,

$$h'(x) = g'(x) - 1.$$

Since

$$g'(-3) = 2.4,$$

we obtain

$$h'(-3) = g'(-3) - 1 = 1.4 > 0.$$

Therefore, $h(x) > 0$ immediately to the right of $x = -3$. Since h has no other zeros on $(-3, 1)$, it follows that

$$h(x) > 0, \quad -3 < x < 1.$$

FPI started sufficiently near the fixed point 2 converges to 2 . By the local convergence condition for fixed-point iteration,

$$|g'(2)| < 1.$$

Hence

$$h'(2) = g'(2) - 1 < 0,$$

so $h(x) > 0$ immediately to the left of $x = 2$.

Since $h(1) = h(2) = 0$ and $h(x) > 0$ on $(1, 2)$, the function h cannot change sign at $x = 1$. Thus $x = 1$ is a tangential zero of h , and therefore

$$h'(1) = 0.$$

Finally,

$$h'(1) = g'(1) - 1 = 0,$$

so

$$g'(1) = 1.$$

Problem 7. Let $f(x) = x^n - ax^{n-1}$, and set $g(x) = x^n$. (a) Use the Sensitivity Formula to give a prediction for the nonzero root of $f(x) = x^n - ax^{n-1} + \epsilon x^n$ for small ϵ . (b) Find the nonzero root and compare with the prediction.

Solution.

$$(a) \quad f(x) = x^{n-1}(x - a),$$

so the nonzero root is $r = a$. By the Sensitivity Formula,

$$\Delta r \approx -\frac{\epsilon g(r)}{f'(r)}.$$

Since

$$g(a) = a^n, \quad f'(a) = a^{n-1},$$

we get

$$\Delta r = -\epsilon a.$$

Therefore,

$$r_{\text{pred}} = a(1 - \epsilon).$$

$$(b) \quad x^n - ax^{n-1} + \epsilon x^n = x^{n-1}((1 + \epsilon)x - a) = 0.$$

Thus the nonzero root is

$$x = \frac{a}{1 + \epsilon}.$$

For small ϵ ,

$$\frac{a}{1 + \epsilon} = a(1 - \epsilon + O(\epsilon^2)),$$

which agrees with the prediction $a(1 - \epsilon)$.

Problem 8. Suppose Newton's Method is applied to the function $f(x) = 1/x$. If the initial guess is $x_0 = 1$, find x_{50} .

Solution The Newton method for this condition is the following:

$$f(x) = 1/x$$

$$f'(x) = -1/x^2$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

$$x_{i+1} = x_i - \frac{1/x_i}{-1/x_i^2} = 2x_i$$

Define $x_0 = 1$, then $x_1 = 2x_0$, $x_2 = 2x_1 = 2 \cdot 2x_0$, ..., $x_{50} = 2^{50}x_0 = 2^{50}$

Problem 9. Apply Newton's Method to find both roots of the function $f(x) = 14xe^{x-2} - 12e^{x-2} - 7x^3 + 20x^2 - 26x + 12$ on the interval $[0, 3]$. For each root, print out the sequence of iterates, the errors e_i , and the relevant error ratio e_{i+1}/e_i^2 or e_{i+1}/e_i that converges to a nonzero limit. Match the limit with the expected value M from Theorem 1.11 or S from Theorem 1.12.

Solution. First we implement Newton's method and find a root.

```

1 import math
2
3 def f(x):
4     return 14*x*math.exp(x-2) - 12*math.exp(x-2) - 7*x**3 + 20*x**2 - 26*x + 12
5
6 def df(x):
7     return 14*math.exp(x-2) + 14*x*math.exp(x-2) - 12*math.exp(x-2) - 21*x**2 + 40*x - 26
8
9
10 def newton(x0, n):
11     x = x0
12
13     print(f"x0 = {x0}")
14
15     for i in range(n):
16         x = x - f(x) / df(x)
17         print(i, x)
18
19 newton(3, 20)
20
21
22 output:
23
24 x0 = 3
25 0 2.7338502571320995
26 1 2.529771874045484
27 2 2.3766658229690543
28 3 2.264145581220574
29 4 2.183030922118089
30 5 2.1255657773577505
31 6 2.0854615043527582
32 7 2.057814950176053
33 8 2.038938028160986
34 9 2.026140999384837
35 10 2.0175105386621683
36 11 2.0117113478603845
37 12 2.007824507284031
38 13 2.0052239304287878
39 14 2.0034860134043493
40 15 2.002325522624122
41 16 2.001551023012529
42 17 2.0010343150515664
43 18 2.000689677414062
44 19 2.0004598408486887
45
46 newton(1, 20)
47 x0 = 1
48 0 0.7627845835848802
49 1 0.840821691829043
50 2 0.8565332220104341
51 3 0.8571419618530481
52 4 0.8571428571409215
53 5 0.857142857142857
54 6 0.8571428571428577
55 7 0.8571428571428563
56 8 0.8571428571428577
57 9 0.8571428571428563
58 10 0.8571428571428577
59 11 0.8571428571428563
60 12 0.8571428571428577
61 13 0.8571428571428563
62 14 0.8571428571428577
63 15 0.8571428571428563

```

```

64 16 0.8571428571428577
65 17 0.8571428571428563
66 18 0.8571428571428577
67 19 0.8571428571428563

```

Checking the root $x=2$ - it is the exact solution. Check the M and S values from theorems and check if they match the error ratio.

```

1 def ddf(x):
2     return 28*math.exp(x-2) + 14*x*math.exp(x-2) \
3         - 12*math.exp(x-2) - 42*x + 40
4
5 def dddf(x):
6     return 30 * math.exp(x - 2) + 14 * x * math.exp(x - 2) - 42
7
8 # print(f"M={ddf(2) / (2 * df(2))}") error as the f'(x) = 0
9
10 print(f(2))
11 print(df(2))
12 print(ddf(2))
13 print(dddf(2))
14
15
16 output:
17
18 0.0
19 0.0
20 0.0
21 16.0
22
23
24 print(f(6/7))
25 print(df(6/7))
26
27
28 output:
29 0.0
30 -2.6781653403215557

```

```

1 def newton_w_errors(x0, n, root=2):
2     x = x0
3     errors = []
4
5     print(f"x0 = {x0}")
6
7     for i in range(n):
8         e = abs(x - root)
9         errors.append(e)
10
11         if i == 0:
12             print(f"i={i}: x={x:.12f}, e={e:.12e}")
13         elif i == 1:
14             ratio_lin = errors[-1] / errors[-2]
15             print(f"i={i}: x={x:.12f}, e={e:.12e}, "
16                 f"e_i/e_(i-1)={ratio_lin:.12e}")
17         else:
18             ratio_lin = errors[-1] / errors[-2]
19             ratio_quad = errors[-1] / errors[-2]**2
20             print(f"i={i}: x={x:.12f}, e={e:.12e}, "
21                 f"e_i/e_(i-1)={ratio_lin:.12e}, "
22                 f"e_i/e_(i-1)^2={ratio_quad:.12e}")
23

```

```

24         x = x - f(x) / df(x)
25
26
27 newton_w_errors(3, 20, 2)
28
29
30 output:
31
32 x0 = 3
33 i=0: x=3.000000000000, e=1.000000000000e+00
34 i=1: x=2.733850257132, e=7.338502571321e-01, e_i/e_(i-1)=7.338502571321e-01
35 i=2: x=2.529771874045, e=5.297718740455e-01, e_i/e_(i-1)=7.219073222320e-01, e_i/e_(i-1)
    ^2=9.837256514063e-01
36 i=3: x=2.376665822969, e=3.766658229691e-01, e_i/e_(i-1)=7.109962635289e-01, e_i/e_(i-1)
    ^2=1.342080050607e+00
37 i=4: x=2.264145581221, e=2.641455812206e-01, e_i/e_(i-1)=7.012730253530e-01, e_i/e_(i-1)
    ^2=1.861790963208e+00
38 i=5: x=2.183030922118, e=1.830309221181e-01, e_i/e_(i-1)=6.929168425697e-01, e_i/e_(i-1)
    ^2=2.623238440589e+00
39 i=6: x=2.125565777358, e=1.255657773578e-01, e_i/e_(i-1)=6.860358670801e-01, e_i/e_(i-1)
    ^2=3.748196529532e+00
40 i=7: x=2.085461504353, e=8.546150435276e-02, e_i/e_(i-1)=6.806114384915e-01, e_i/e_(i-1)
    ^2=5.420357782299e+00
41 i=8: x=2.057814950176, e=5.781495017605e-02, e_i/e_(i-1)=6.765028373174e-01, e_i/e_(i-1)
    ^2=7.915877943419e+00
42 i=9: x=2.038938028161, e=3.893802816099e-02, e_i/e_(i-1)=6.734941056321e-01, e_i/e_(i-1)
    ^2=1.164913406621e+01
43 i=10: x=2.026140999385, e=2.614099938484e-02, e_i/e_(i-1)=6.713488232316e-01, e_i/e_(i-1)
    ^2=1.724146945644e+01
44 i=11: x=2.017510538662, e=1.751053866217e-02, e_i/e_(i-1)=6.698496260371e-01, e_i/e_(i-1)
    ^2=2.562448421255e+01
45 i=12: x=2.011711347860, e=1.171134786038e-02, e_i/e_(i-1)=6.688171098749e-01, e_i/e_(i-1)
    ^2=3.819511910961e+01
46 i=13: x=2.007824507284, e=7.824507284031e-03, e_i/e_(i-1)=6.681133015012e-01, e_i/e_(i-1)
    ^2=5.704836962116e+01
47 i=14: x=2.005223930429, e=5.223930428788e-03, e_i/e_(i-1)=6.676369820051e-01, e_i/e_(i-1)
    ^2=8.532639280274e+01
48 i=15: x=2.003486013404, e=3.486013404349e-03, e_i/e_(i-1)=6.673162002960e-01, e_i/e_(i-1)
    ^2=1.277421683525e+02
49 i=16: x=2.002325522624, e=2.325522624122e-03, e_i/e_(i-1)=6.671008841276e-01, e_i/e_(i-1)
    ^2=1.913649796341e+02
50 i=17: x=2.001551023013, e=1.551023012529e-03, e_i/e_(i-1)=6.669567504700e-01, e_i/e_(i-1)
    ^2=2.867986505707e+02
51 i=18: x=2.001034315052, e=1.034315051566e-03, e_i/e_(i-1)=6.668599003441e-01, e_i/e_(i-1)
    ^2=4.299484243350e+02
52 i=19: x=2.000689677414, e=6.896774140621e-04, e_i/e_(i-1)=6.667962658164e-01, e_i/e_(i-1)
    ^2=6.446742361591e+02
53
54
55 newton_w_errors(1, 20, 6/7)
56
57 x0 = 1
58 i=0: x=1.000000000000, e=1.428571428571e-01
59 i=1: x=0.762784583585, e=9.435827355798e-02, e_i/e_(i-1)=6.605079149058e-01
60 i=2: x=0.840821691829, e=1.632116531381e-02, e_i/e_(i-1)=1.729701561759e-01, e_i/e_(i-1)
    ^2=1.833121247917e+00
61 i=3: x=0.856533222010, e=6.096351324230e-04, e_i/e_(i-1)=3.735242678456e-02, e_i/e_(i-1)
    ^2=2.288588226782e+00
62 i=4: x=0.857141961853, e=8.952898089554e-07, e_i/e_(i-1)=1.468566625085e-03, e_i/e_(i-1)
    ^2=2.408927154916e+00
63 i=5: x=0.857142857141, e=1.935562821131e-12, e_i/e_(i-1)=2.161939968232e-06, e_i/e_(i-1)
    ^2=2.414793451916e+00
64 i=6: x=0.857142857143, e=1.110223024625e-16, e_i/e_(i-1)=5.735918320523e-05, e_i/e_(i-1)

```

```

65 i=7: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=5.000000000000e+00, e_i/e_(i-1)
    ^2=4.503599627370e+16
66 i=8: x=0.857142857143, e=7.771561172376e-16, e_i/e_(i-1)=1.400000000000e+00, e_i/e_(i-1)
    ^2=2.522015791327e+15
67 i=9: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=7.142857142857e-01, e_i/e_(i-1)
    ^2=9.191019647695e+14
68 i=10: x=0.857142857143, e=7.771561172376e-16, e_i/e_(i-1)=1.400000000000e+00, e_i/e_(i-1)
    ^2=2.522015791327e+15
69 i=11: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=7.142857142857e-01, e_i/e_(i-1)
    ^2=9.191019647695e+14
70 i=12: x=0.857142857143, e=7.771561172376e-16, e_i/e_(i-1)=1.400000000000e+00, e_i/e_(i-1)
    ^2=2.522015791327e+15
71 i=13: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=7.142857142857e-01, e_i/e_(i-1)
    ^2=9.191019647695e+14
72 i=14: x=0.857142857143, e=7.771561172376e-16, e_i/e_(i-1)=1.400000000000e+00, e_i/e_(i-1)
    ^2=2.522015791327e+15
73 i=15: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=7.142857142857e-01, e_i/e_(i-1)
    ^2=9.191019647695e+14
74 i=16: x=0.857142857143, e=7.771561172376e-16, e_i/e_(i-1)=1.400000000000e+00, e_i/e_(i-1)
    ^2=2.522015791327e+15
75 i=17: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=7.142857142857e-01, e_i/e_(i-1)
    ^2=9.191019647695e+14
76 i=18: x=0.857142857143, e=7.771561172376e-16, e_i/e_(i-1)=1.400000000000e+00, e_i/e_(i-1)
    ^2=2.522015791327e+15
77 i=19: x=0.857142857143, e=5.551115123126e-16, e_i/e_(i-1)=7.142857142857e-01, e_i/e_(i-1)
    ^2=9.191019647695e+14

```

At $x = 2$, we have

$$f(2) = 0, \quad f'(2) = 0, \quad f''(2) = 0,$$

while

$$f'''(2) = 16 \neq 0.$$

Therefore, $x = 2$ is a root of multiplicity

$$m = 3.$$

Since the root is multiple, the quadratic convergence result for a simple root does not apply. Instead, by Theorem 1.12, Newton's method converges linearly with asymptotic error ratio

$$S = \frac{m-1}{m}.$$

For $m = 3$,

$$S = \frac{3-1}{3} = \frac{2}{3} \approx 0.6666667.$$

From the computed iterates, the ratios

$$\frac{e_{i+1}}{e_i}$$

approach

$$0.6666667,$$

Thus, the numerical result agrees with the theoretical value:

$$S = \frac{2}{3}.$$

In this problem Newton's method converges linearly for the triple root $x = 2$, rather than quadratically as it would for a simple root.

For another root $x=6/7$ the ratio $e_i/e_{i-1} \rightarrow 0$, and $e_i/e_{i-1}^2 \rightarrow M$ and the theoretical M is

$$M = \frac{f''(r)}{2f'(r)} \approx -2.4138509$$

and from code the ration converges to this value (magnitude of M) after 4-5 iterations and then the ratio becomes extremly big. So for this initial value algorithm converges quadratically.

Problem 10. Sauer, Exercise 4a, page 85 Solve the system by finding the LU factorization and then carrying out the two-step back substitution.

Solution. We have

$$A = \begin{pmatrix} 3 & 1 & 2 \\ 6 & 3 & 4 \\ 3 & 1 & 5 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 1 \\ 3 \end{pmatrix}.$$

We start with

$$A = \begin{pmatrix} 3 & 1 & 2 \\ 6 & 3 & 4 \\ 3 & 1 & 5 \end{pmatrix}.$$

Using Gaussian elimination, we first obtain U .

For the first elimination step,

$$R_2 \leftarrow R_2 - 2R_1,$$

so

$$\begin{pmatrix} 3 & 1 & 2 \\ 6 & 3 & 4 \\ 3 & 1 & 5 \end{pmatrix} \longrightarrow \begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 0 \\ 3 & 1 & 5 \end{pmatrix}.$$

The elimination multiplier is

$$l_{21} = 2.$$

Next,

$$R_3 \leftarrow R_3 - R_1,$$

and therefore

$$\begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 0 \\ 3 & 1 & 5 \end{pmatrix} \longrightarrow \begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

Thus,

$$l_{31} = 1,$$

and

$$U = \begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

The corresponding elementary elimination matrices are

$$E_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad E_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}.$$

Hence,

$$U = E_2 E_1 A.$$

Taking the inverses gives

$$A = E_1^{-1} E_2^{-1} U.$$

Since

$$E_1^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad E_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix},$$

we obtain

$$L = E_1^{-1} E_2^{-1}.$$

Therefore,

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

Thus the LU factorization is

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

Then

$$LUx = b.$$

First, let

$$Ux = y.$$

Then we solve

$$Ly = b :$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 3 \end{pmatrix}.$$

Thus,

$$y_1 = 0, \quad y_2 = 1, \quad y_3 = 3.$$

Next, solve

$$Ux = y :$$

$$\begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 3 \end{pmatrix}.$$

Back substitution gives

$$x_3 = 1,$$

$$x_2 = 1,$$

$$x_1 = -1.$$

Therefore,

$$x = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}$$

Problem 11. Sauer, Exercise 7, page 85 Assume that your computer can solve 1000 problems of type $Ux = c$, where U is an upper-triangular 500×500 matrix, per second. Estimate how long it will take to solve a full 5000×5000 matrix problem $Ax = b$. Answer in minutes and seconds.

Solution. For an upper-triangular 500×500 system, the cost is approximately

$$\frac{500^2}{2} * 2 = 250000$$

operations.

The computer solves 1000 such systems per second, so its speed is

$$1000 \cdot 250000 = 2.5 * 10^8$$

operations per second.

For a full 5000×5000 system, Gaussian elimination costs

$$\frac{2(5000)^3}{3}$$

operations.

Therefore,

$$t = \frac{\frac{2(5000)^3}{3}}{2.5 * 10^8} = 333.3 \text{ seconds.}$$

Converting to minutes and seconds,

$$333.3 \text{ seconds} = 5 \text{ minutes } 33 \text{ seconds.}$$

Problem 13. This problem explores the relationship between condition number and the volume of a random parallelepiped in 4-dimensional space whose sides are unit vectors. Let a_1, a_2, a_3, a_4 be vectors which emanate from a given vertex of the parallelepiped and describe its sides. By a well-known result from geometry, the volume of the parallelepiped is then $|\det([a_1, a_2, a_3, a_4])|$. Your program for this problem should do the following:

(a) Generate 4 random column vectors of size 4-by-1, viewing them as the 4 columns of a 4-by-4 random matrix A .

(b) Normalize the columns to have length 1.

(c) Compute the volume of the parallelepiped defined by the normalized vectors. Your program must use the LU factorization for computing the determinant (that is do not use ready command like `det(A)` here).

(d) Compute the condition number $\kappa(A)$ with any available or written by you function `cond(A)`.

(e) Repeat the process 1000 times, and produce a scatter plot of $|\det(A)|$ versus $1/\kappa(A)$.

What are the maximum and minimum values found for $|\det(A)|$? What is the condition number corresponding to each of these values? How can you explain the extreme values of the condition number and the corresponding values of the determinant?

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 def det_LU(A):
6     A = A.copy()
7     n = A.shape[0]
8
9     for k in range(n - 1):
10         for i in range(k + 1, n):

```

```

11         m = A[i, k] / A[k, k]
12         A[i, k:] = A[i, k:] - m * A[k, k:]
13
14     return np.prod(np.diag(A))
15
16 det_values = []
17 inv_cond_values = []
18
19 for i in range(1000):
20
21     A = np.random.rand(4, 4)
22     A = A / np.linalg.norm(A, axis=0)
23
24     det_A = det_LU(A)
25
26     cond_A = np.linalg.cond(A)
27
28     det_values.append(abs(det_A))
29     inv_cond_values.append(1 / cond_A)
30
31 plt.scatter(inv_cond_values, det_values)
32 plt.xlabel(r'$1/\kappa(A)$')
33 plt.ylabel(r'$|\det(A)|$')
34 plt.show()

```

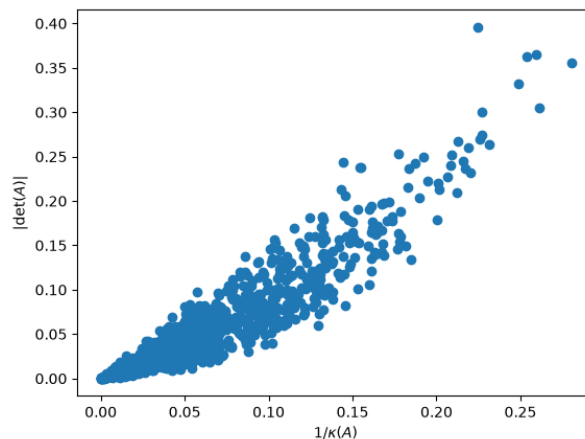


Рис. 2: $1/\kappa(A)$ vs $|\det(A)|$

min. $\det = 3.181008920295286e-05$, and corresponding $1/k = 7.545382662272349e-05$ and $k \approx 1.33 \cdot 10^4$,
max. $\det = 0.396122966318929$, and corresponding $1/k = 0.2248095617476819$ and $k \approx 4.45$

The extreme values can be explained by the geometry of the vectors. The closer the vectors are to each other, the closer the volume of the parallelepiped, and therefore ($|\det(A)|$), is to zero. In this case, the matrix is close to singular and its condition number becomes large. Conversely, when the vectors are more independent, the determinant is larger and the condition number is smaller. So the determinant and the condition number have an opposite relationship.

Problem 14. Find the relative forward and backward errors and error magnification factor for the following approximate solutions of the system $x_1 + 2x_2 = 3$, $2x_1 + 4.01x_2 = 6.01$: (a) $[-10, 6]$ (c) $[-600, 301]$ (e) What is the condition number of the coefficient matrix?

Solution.

$$(a) \quad \hat{x} = \begin{pmatrix} -10 \\ 6 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The relative forward error is

$$\frac{\|\hat{x} - x\|_{\infty}}{\|x\|_{\infty}} = \frac{\max(|-10 - 1|, |6 - 1|)}{\max(|1|, |1|)} = \frac{11}{1} = 11.$$

The residual is

$$b - A\hat{x} = \begin{pmatrix} 3 \\ 6.01 \end{pmatrix} - \begin{pmatrix} 1 & 2 \\ 2 & 4.01 \end{pmatrix} \begin{pmatrix} -10 \\ 6 \end{pmatrix} = \begin{pmatrix} 1 \\ 1.95 \end{pmatrix}.$$

The relative backward error is

$$\frac{\|b - A\hat{x}\|_{\infty}}{\|b\|_{\infty}} = \frac{\max(1, 1.95)}{\max(3, 6.01)} = \frac{1.95}{6.01} \approx 0.32446.$$

The error magnification factor is

$$\frac{11}{0.32446} \approx 33.90.$$

Forward error = 11, Backward error ≈ 0.32446 , Magnification factor ≈ 33.90

$$(c) \quad = \begin{pmatrix} -600 \\ 301 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The relative forward error is

$$\frac{\|\hat{x} - x\|_{\infty}}{\|x\|_{\infty}} = \frac{\max(|-600 - 1|, |301 - 1|)}{\max(|1|, |1|)} = \frac{601}{1} = 601.$$

The residual is

$$b - A\hat{x} = \begin{pmatrix} 3 \\ 6.01 \end{pmatrix} - \begin{pmatrix} 1 & 2 \\ 2 & 4.01 \end{pmatrix} \begin{pmatrix} -600 \\ 301 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

The relative backward error is

$$\frac{\|b - A\hat{x}\|_{\infty}}{\|b\|_{\infty}} = \frac{\max(1, 1)}{\max(3, 6.01)} = \frac{1}{6.01} \approx 0.16639.$$

The error magnification factor is

$$\frac{601}{0.16639} \approx 3612.$$

Forward error = 601, Backward error ≈ 0.16639 , Magnification factor 3612

(e) Condition number of the coefficient matrix A.

The coefficient matrix is

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 4.01 \end{pmatrix}.$$

We compute the condition number in the $\|\cdot\|_{\infty}$ norm:

$$\kappa_{\infty}(A) = \|A\|_{\infty} \cdot \|A^{-1}\|_{\infty}.$$

$$\|A\|_{\infty} = \max(|1| + |2|, |2| + |4.01|) = \max(3, 6.01) = 6.01.$$

$$\det A = (1)(4.01) - (2)(2) = 4.01 - 4 = 0.01.$$

$$A^{-1} = \frac{1}{\det A} \begin{pmatrix} 4.01 & -2 \\ -2 & 1 \end{pmatrix} = \frac{1}{0.01} \begin{pmatrix} 4.01 & -2 \\ -2 & 1 \end{pmatrix} = 100 \begin{pmatrix} 4.01 & -2 \\ -2 & 1 \end{pmatrix} = \begin{pmatrix} 401 & -200 \\ -200 & 100 \end{pmatrix}.$$

$$\|A^{-1}\|_{\infty} = \max(|401| + |-200|, |-200| + |100|) = \max(601, 300) = 601.$$

$$\kappa_{\infty}(A) = \|A\|_{\infty} \cdot \|A^{-1}\|_{\infty} = 6.01 \cdot 601 = 3612.01.$$

Thus, the condition number of the coefficient matrix is

$$\kappa_{\infty}(A) = 3612.01.$$

Problem 16. Rearrange the equations to form a strictly diagonally dominant system. Apply two steps of the Jacobi and Gauss-Seidel Methods from starting vector $[0, \dots, 0]$.

$$c) \begin{pmatrix} 4 & 0 & 1 \\ 1 & 2 & 0 \\ 0 & 3 & 4 \end{pmatrix}$$

Solution.

Jacobi:

$$\begin{aligned} x^{(0)} &= (v, w, u) = (0, 0, 0) \\ v^{(k+1)} &= \frac{5 - u^{(k)}}{4} \\ w^{(k+1)} &= \frac{2 - v^{(k)}}{2} \\ u^{(k+1)} &= \frac{-3w^{(k)}}{4} \end{aligned}$$

First iteration:

$$\begin{aligned} v^{(1)} &= \frac{5}{4} \\ w^{(1)} &= 1 \\ u^{(1)} &= 0 \end{aligned}$$

Second iteration:

$$\begin{aligned} v^{(2)} &= \frac{5}{4} \\ w^{(2)} &= 0.375 \\ u^{(2)} &= -0.75 \end{aligned}$$

Gauss-Seidel method:

$$\begin{aligned} x^{(0)} &= (v, w, u) = (0, 0, 0) \\ v^{(k+1)} &= \frac{5 - u^{(k)}}{4} \\ w^{(k+1)} &= \frac{2 - v^{(k+1)}}{2} \\ u^{(k+1)} &= \frac{-3w^{(k+1)}}{4} \end{aligned}$$

First iteration:

$$\begin{aligned} v^{(1)} &= \frac{5}{4} \\ w^{(1)} &= 0.375 \\ u^{(1)} &= -3 * 0.375 / 4 = -0.28125 \end{aligned}$$

Second iteration:

$$\begin{aligned} v^{(2)} &= \frac{5.28125}{4} = 1.3203125 \\ w^{(2)} &= \frac{2 - 1.3203125}{2} = 0.33984375 \\ u^{(2)} &= \frac{3 * 0.33984375}{4} = -0.2548828125 \end{aligned}$$

Problem 17. Carry out the steps of Computer Problem 1 with $n = 100$ for Gauss-Seidel Method

```
1 import numpy as np
2
```

```

3
4 def row(i):
5     if i == 0:
6         return np.array([3, -1] + [0] * 98)
7     elif i < 99:
8         return np.array([0] * (i - 1) + [-1, 3, -1] + [0] * (98 - i))
9     return np.array([0] * 98 + [-1, 3])
10
11
12 A = np.array([row(i) for i in range(100)])
13
14 x_exact = np.ones(100)
15
16 b = A @ x_exact
17
18
19 def Gauss_Seidel(A, b, x0, n):
20     x = x0.copy()
21
22     for k in range(n):
23         for i in range(len(b)):
24             s1 = np.dot(A[i, :i], x[:i])
25
26             s2 = np.dot(A[i, i+1:], x[i+1:])
27
28             x[i] = (b[i] - s1 - s2) / A[i, i]
29
30     return x
31
32
33 Gauss_Seidel(A, b, np.zeros(100, dtype=float), 30)
34
35
36 output:
37 array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
38        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
39        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
40        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
41        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
42        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
43
44 Gauss_Seidel(A, b, np.zeros(100, dtype=float), 20)
45
46
47 output:
48 array([0.99999952, 0.99999928, 0.99999916, 0.9999991 , 0.99999908,
49        0.99999906, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
50        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
51        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
52        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
53        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
54        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
55        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
56        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
57        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
58        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
59        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
60        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
61        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
62        0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905, 0.99999905,
63        0.99999905, 0.99999905, 0.99999906, 0.99999907, 0.99999911,
64        0.99999916, 0.99999924, 0.99999933, 0.99999944, 0.99999954,

```

```

66 0.999999965, 0.999999974, 0.999999981, 0.999999987, 0.999999992,
67 0.999999995, 0.999999997, 0.999999999, 0.999999999, 1.  ])
```

Problem 18. How many floating point operations does it take to compute 1. the product Ax , where A is the sparse $n \times n$ matrix in Problem 17? 2. the product Bx , where B is a full $n \times n$ matrix? In both cases assume x is an $n \times 1$ column vector.

Solution.

1. 496 (3 muls and 2 adds for 98 rows + 2 muls and 1 add for 2 rows)
2. $2n^2 - n$ (n muls and $n - 1$ adds for each row)

Problem 19. This problem concerns polynomial interpolation based on expansion

$$p(x) = c_1 + c_2x + c_3x^2 + \dots + c_nx^{n-1}$$

in the monomial basis in tandem with the Vandermonde approach.

- (a) Assume that the data $(x_k, y_k) : k = 1, \dots, n$ is given, where the nodes x_k are distinct.
- (b) Write a function that returns the vector of coefficients $c = (c_1, \dots, c_n)^T$ defining the interpolant, given input vectors $x = (x_1, \dots, x_n)^T$ and $y = (y_1, \dots, y_n)^T$. Use your function to plot the example we have done in class for the data set $D_4 = (0, 1); (1, 4); (2, 1); (3, 1)$.
- (c) Find the infinity-norm condition number of the matrix V for $n = 5, 10, 15, 20$ equally spaced points on the interval $[0, 1]$.

Solution.

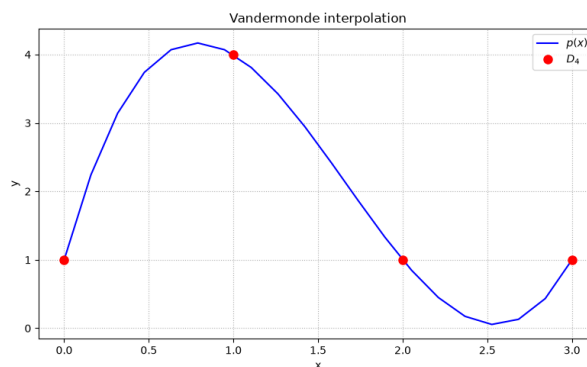
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def Vandermond(x: np.ndarray) -> np.ndarray:
5     n = len(x)
6     return np.array([[x[i]**j for j in range(n)] for i in range(n)])
7
8 def get_coeffs(x: np.ndarray, y: np.ndarray) -> np.ndarray:
9     V = Vandermond(x)
10    return np.linalg.solve(V, y)
11
12
13 x = np.array([0, 1, 2, 3], dtype=float)
14 y = np.array([1, 4, 1, 1], dtype=float)
15
16 coeffs = get_coeffs(x, y)
17 print(coeffs)
18 print("p(x_k):", Vandermond(x) @ coeffs)
19 print("y_k: ", y)
```

Output:

```

[ 1.  9. -7.5  1.5]
p(x_k): [1.  4.  1.  1.]
y_k:    [1.  4.  1.  1.]
```



To get the condition number for each case, we use the code:
(c)

```

1 for n in [5, 10, 15, 20]:
2     points = np.linspace(0, 1, n)
3     V = Vandermonde(points)
4     condition_number = np.linalg.cond(V, np.inf)
5     print(f"Condition number for {n} points: {condition_number}")
6
7 output:
8
9 Condition number for 5 points: 1706.6666666666665
10 Condition number for 10 points: 48183984.002129205
11 Condition number for 15 points: 1605542347896.6553
12 Condition number for 20 points: 4.984423385657521e+16

```

The higher a power of polynomial, the higher condition number is, so the error is bigger.

Problem 20. 3.1.1 (c) Use Lagrange interpolation to find a polynomial that passes through the points
(c) (0,-2), (2,1), (4,4)

3.1.2 Use Newton's divided differences to find the interpolating polynomials of the points in Exercise 1,
and verify agreement with the Lagrange interpolating polynomial.

3.1.3 (c) How many degree d polynomials pass through the four points (-1,3), (1,1), (2,3), (3,7)? Write
one down if possible. (c) d = 6.

Solution. 3.1.1 (c)

$$L_i = \frac{(x-x_1)(x-x_2)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)}{(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)}$$

$$L_0 = \frac{(x-2)(x-4)}{(0-2)(0-4)} = \frac{(x-2)(x-4)}{8}$$

$$L_1 = \frac{(x-0)(x-4)}{(2-0)(2-4)} = \frac{x(x-4)}{-4}$$

$$L_2 = \frac{(x-0)(x-2)}{(4-0)(4-2)} = \frac{x(x-2)}{8}$$

$$p(x) = y_0 \cdot L_0 + y_1 \cdot L_1 + y_2 \cdot L_2 = -\frac{1}{4}(x-2)(x-4) - \frac{x(x-4)}{4} + \frac{1}{2}x(x-2) = \frac{3}{2}x - 2$$

3.1.2

$$0 \mid -2$$

$$\frac{1 - (-2)}{2 - 0} = \frac{3}{2}$$

$$2 \mid 1$$

$$\frac{\frac{3}{2} - \frac{3}{2}}{4 - 0} = 0$$

$$\frac{4 - 1}{4 - 2} = \frac{3}{2}$$

$$4 \mid 4$$

$$p(x) = -2 + 3/2(x - 0) = \frac{3}{2}x - 2$$

The polynomial is the same as in the Lagrange method.

3.1.3 (c) To find coefficients, we have to solve the system of 4 equations with 7 variables. The rank of
such matrix is less than the quantity of variables, so the number of the solutions is infinite.

We can define the view of polynomial as a sum of partial solution and homogenous:

$$(x+1)(x-1)(x-2)(x-3) * (ax^2 + bx + c) + f_{part}(x) = (x+1)(x-1)(x-2)(x-3) * (ax^2 + bx + c) + (x^2 - x + 1)$$

Problem 21. Write a function that computes coefficients of Newton's interpolant along with your own
routine for evaluation (using nested polynomial method, Horner's rule) to find and plot the polynomial
that is zero at $x = 1, 2, 4, 5, \dots, 10, 11$ and satisfies $p(3) = 1$. Note that this is the Lagrange's basis

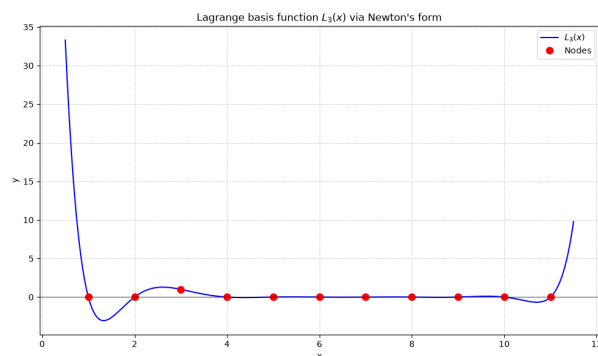
function $L_3(x)$ for the set of nodal points $\{x_j\}_{j=1}^{11} = 1, 2, 3, \dots, 11$.

Solution. The function and the interpolant:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 def newton_coeffs(x: np.ndarray, y: np.ndarray) -> np.ndarray:
6     n = len(x)
7     c = y.astype(float).copy()
8     for j in range(1, n):
9         for i in range(n - 1, j - 1, -1):
10             c[i] = (c[i] - c[i - 1]) / (x[i] - x[i - j])
11     return c
12
13
14 def newton_eval(x_nodes: np.ndarray, c: np.ndarray, x_eval: float) -> float:
15     result = c[-1]
16     for i in range(len(c) - 2, -1, -1):
17         result = result * (x_eval - x_nodes[i]) + c[i]
18     return result
19
20
21 x = np.arange(1, 12, dtype=float)
22 y = np.zeros(11)
23 y[2] = 1.0
24
25 c = newton_coeffs(x, y)
26 print("Newton coefficients:", c)
27
28
29 p3 = newton_eval(x, c, 3.0)
30 print(f"p(3) = {p3}")
31
32
33 x_fine = np.linspace(0.5, 11.5, 500)
34 y_fine = np.array([newton_eval(x, c, xi) for xi in x_fine])
35
36 plt.figure(figsize=(10, 6))
37 plt.plot(x_fine, y_fine, 'b-', label=r'$L_3(x)$')
38 plt.plot(x, y, 'ro', markersize=8, label='Nodes')
39 plt.axhline(y=0, color='k', linewidth=0.5)
40 plt.xlabel('x')
41 plt.ylabel('y')
42 plt.title("Lagrange basis function $L_3(x)$ via Newton's form")
43 plt.grid(True, ls=':')
44 plt.legend()
45 plt.tight_layout()
46 plt.show()

```



Problem 22. Use your routines from Problem 19 to interpolate the data sets $(x_j, y_j) : j = 1, \dots, n$, where $y_j = f(x_j)$

- with $f(x) = 1/(x^2 + 1)$.
1. $x_j = 4 + 8 \frac{j}{n-1}$, $n = 11$.
 2. $x_j = 4 \cos \frac{\pi(2j-1)}{2n}$, $n = 11$.

In each case, return 3 plots: 1). The data and the interpolant on $[4, 4]$. The interpolants should be plotted on a dense collection of points, say 500 equally spaced points. Include on your plots markers of some kind that illustrate the data points.

2) The error $e(x) = f(x)p(x)$, for $x \in [4, 4]$

3) The function $g(x) = \frac{1}{n!} \prod_{i=1}^n (x - x_k)$ for $x \in [4, 4]$.

Solution. The function from Problem19 is for Vandermonde approach.

```

1 import math
2
3 def f(x):
4     return 1 / (x ** 2 + 1)
5
6 def g(x, x_nodes: np.ndarray):
7     return (np.prod([x - x_node for x_node in x_nodes], axis=0))/ math.factorial(len(x_nodes))
8
9
10 # case 1
11
12 n = 11
13
14 x = np.array([-4 + 8 * j / (n - 1) for j in range(0, n)], dtype=float)
15 y = f(x)
16 coeffs = get_coeffs(x, y)
17 print("coeffs:", coeffs)
18
19 print("p(x_k):", Vandermonde(x) @ coeffs)
20 print("y_k: ", y)
21
22
23 x_fine = np.linspace(-4, 4, 500)
24 y_fine = np.array([
25     sum(coeffs[j] * xi**j for j in range(len(coeffs)))
26     for xi in x_fine
27 ])
28
29 plt.figure(figsize=(10, 6))
30 plt.plot(x_fine, y_fine, 'b-', label=r'$p(x)$')
31 plt.plot(x_fine, f(x_fine), 'g--', alpha=0.5, label=r'$f(x)$')
32 plt.plot(x, y, 'ro', markersize=8, label=r'$D_{11}$')
33 plt.xlabel('x')
34 plt.ylabel('y')
35 plt.title('Vandermonde interpolation, n=11')
36 plt.grid(True, ls=':')
37 plt.legend()
38 plt.tight_layout()
39 plt.show()
40
41
42 error = f(x_fine) - y_fine
43
44 plt.figure(figsize=(10, 6))
45 plt.plot(x_fine, error, 'b-', label=r'$e(x)$')
46 plt.xlabel('x')
47 plt.ylabel('e(x)')
48 plt.title('Error of interpolation')
49 plt.grid(True, ls=':')
50 plt.legend()
51 plt.tight_layout()
52 plt.show()
53

```

```

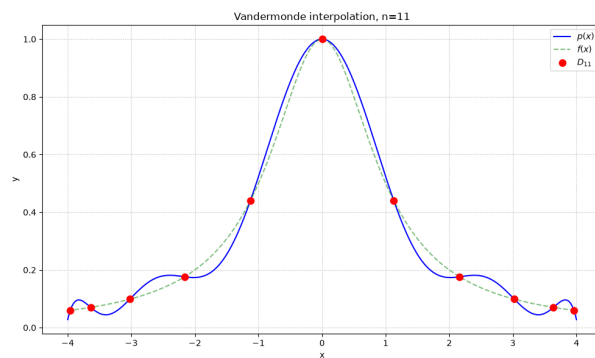
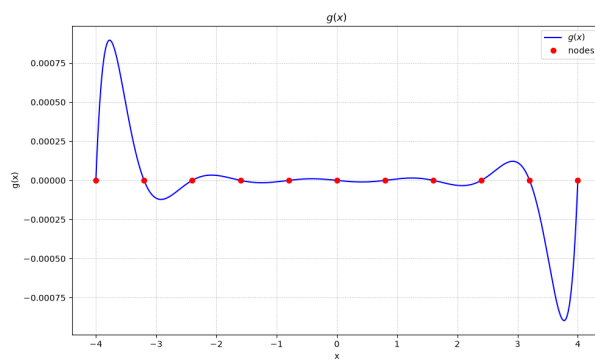
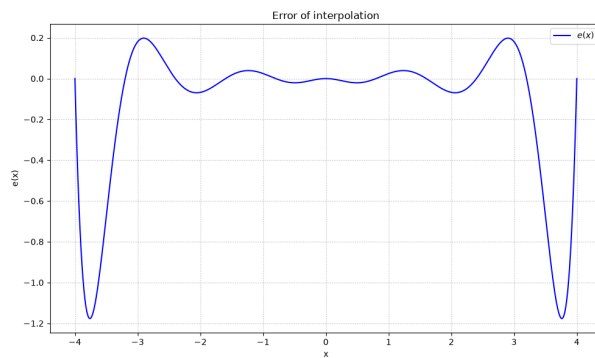
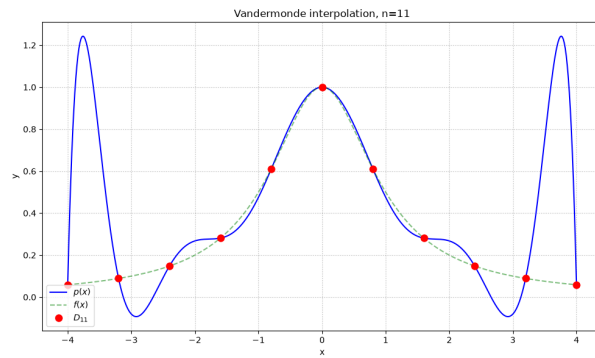
54 plt.figure(figsize=(10, 6))
55 plt.plot(x_fine, g(x_fine, x), 'b-', label=r'$g(x)$')
56 plt.plot(x, np.zeros_like(x), 'ro', markersize=6, label='nodes')
57 plt.xlabel('x')
58 plt.ylabel('g(x)')
59 plt.title('Lagrange basis function $g(x)$')
60 plt.grid(True, ls=':')
61 plt.legend()
62 plt.tight_layout()
63 plt.show()
64
65
66 # case 2
67 import numpy as np
68
69 x = [4 * np.cos(np.pi * (2 * j - 1) / (2 * n)) for j in range(1, n + 1)]
70 y = [f(x) for x in x]
71 coeffs = get_coeffs(x, y)
72 print(coeffs)
73
74
75 print("coeffs:", coeffs)
76
77 print("p(x_k):", Vandermond(x) @ coeffs)
78 print("y_k: ", y)
79
80
81 x_fine = np.linspace(-4, 4, 500)
82 y_fine = np.array([
83     sum(coeffs[j] * xi**j for j in range(len(coeffs)))
84     for xi in x_fine
85 ])
86
87 plt.figure(figsize=(10, 6))
88 plt.plot(x_fine, y_fine, 'b-', label=r'$p(x)$')
89 plt.plot(x_fine, f(x_fine), 'g--', alpha=0.5, label=r'$f(x)$')
90 plt.plot(x, y, 'ro', markersize=8, label=r'$D_{11}$')
91 plt.xlabel('x')
92 plt.ylabel('y')
93 plt.title('Vandermonde interpolation, n=11')
94 plt.grid(True, ls=':')
95 plt.legend()
96 plt.tight_layout()
97 plt.show()
98
99
100 error = f(x_fine) - y_fine
101
102 plt.figure(figsize=(10, 6))
103 plt.plot(x_fine, error, 'b-', label=r'$e(x)$')
104 plt.xlabel('x')
105 plt.ylabel('e(x)')
106 plt.title('Error of interpolation')
107 plt.grid(True, ls=':')
108 plt.legend()
109 plt.tight_layout()
110 plt.show()
111
112
113 plt.figure(figsize=(10, 6))
114 plt.plot(x_fine, g(x_fine, x), 'b-', label=r'$g(x)$')
115 plt.plot(x, np.zeros_like(x), 'ro', markersize=6, label='nodes')
116 plt.xlabel('x')

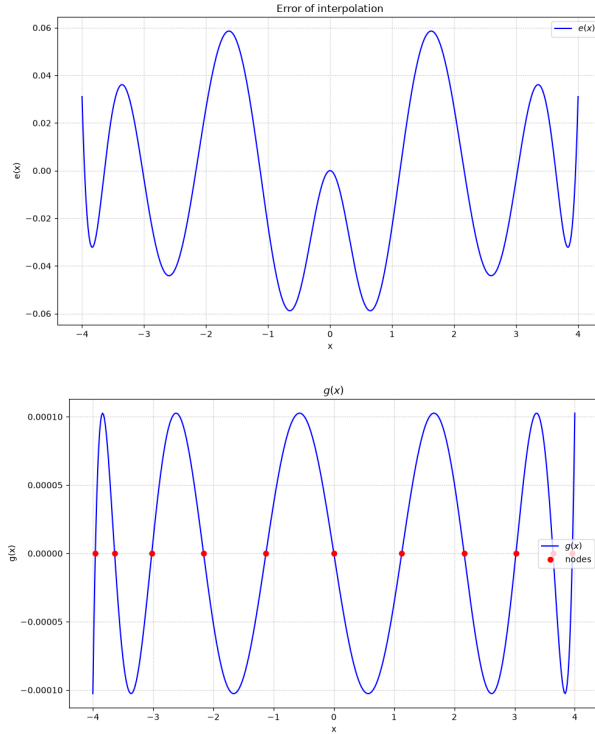
```

```

117 plt.ylabel('g(x)')
118 plt.title('Lagrange basis function $g(x)$')
119 plt.grid(True, ls=':')
120 plt.legend()
121 plt.tight_layout()
122 plt.show()

```





As can be seen from the plots, in the second case the interpolation error does not exceed 0.06, and the interpolant closely follows the plots, in the first case the error is much larger. This improvement is due to the choice of interpolation nodes, which directly controls the magnitude of $\max \omega$ values. This behavior is related to the Runge phenomenon - for this type of function high-order derivatives grow rapidly. When using uniformly spaced nodes, the term $\omega_n(x)$ also becomes large near the endpoints of the interval, leading to a substantial increase in the error bound. In contrast, the Chebyshev nodes (used in the second case) are clustered toward the edges of the interval; this clustering keeps $|\omega(x)|$ relatively uniform and prevents the error from blowing up, even though the high-order derivatives remain large.

Problem 23. Consider the interpolating polynomial for $f(x) = 1/(x+5)$ with interpolation nodes $x = 0, 2, 4, 6, 8, 10$. Find an upper bound for the interpolation error at (a) $x = 1$ and (b) $x = 5$.

Solution. The upper bound for the interpolation error is:

$$|e(x)| \leq \frac{\max |f^{(n)}(\psi)|}{n!} |\omega_n(x)|, \text{ for } \omega = \prod_{i=1}^6 (x - x_i)$$

$$f^{(6)}(x) = \frac{720}{(x+5)^7}$$

as the interval $[0, 10]$ contains all nodes and $x=1$ and $x=5$, maximum value of the derivative for both cases is $720/(0+5)^7$

(a)

$$x = 1 :$$

$$\omega = \prod_{i=1}^6 (1 - x_i) = (1 - 0)(1 - 2)(1 - 4)(1 - 6)(1 - 8)(1 - 10) = -945$$

$$|e(x)| \leq \frac{720/5^7}{720} |945| = \frac{945}{5^7} = \frac{189}{15625}$$

(b)

$$x = 5 :$$

$$\omega = \prod_{i=1}^6 (5 - x_i) = (5 - 0)(5 - 2)(5 - 4)(5 - 6)(5 - 8)(5 - 10) = -225$$

$$|e(x)| \leq \frac{720/5^7}{720} |225| = \frac{225}{5^7} = \frac{9}{3125}$$